Cloudity Suite — guide intégration

Cloudity Suite — rapport d’intégration (lisible)

Date : 2026-09-15 (mise à jour soir)

1. Pourquoi le lien PDF donnait 404

La release GitHub `suite-integration-guide-20260915` était en **brouillon (draft)**. Les drafts ne sont pas téléchargeables publiquement → page 404.

Corrigé : release **publiée**. Lien direct PDF :

https://github.com/PavelDelhomme/Cloudity/releases/download/suite-integration-guide-20260915/cloudity-suite-integration-20260915-184002.pdf

Page release :

https://github.com/PavelDelhomme/Cloudity/releases/tag/suite-integration-guide-20260915

---

2. Restructure déjà faite (et c’est normal)

```
Cloudity/
├── Cloudity.code-workspace
├── products/
│   ├── jobbing-track/     ← submodule COMPLET (= repo JobbingTrack)
│   ├── fuel/README.md     ← placeholder Gasoil (pas encore submodule)
│   └── music/README.md    ← placeholder PLM (pas encore submodule)
├── platform/
│   └── identity-sdk/      ← squelette SSO
├── backend/ frontend/ …   ← cœur Cloudity inchangé
└── docs/cursor/
    ├── BRIEF-INTEGRATION-SUITE.md
    └── COMMENT-OUVRIR-CURSOR.md
```

Pourquoi seulement JobbingTrack dans products/ ?
On intègre **un produit à la fois** pour ne pas risquer les volumes Gasoil (~9 Go) et YTMusic (~21 Go). JT est le pilote.

Le code JobbingTrack sous `products/jobbing-track` **est bien le projet complet** (backend, frontend, mobile, docs…) — ce n'est pas un stub.

---

3. Comment ouvrir dans Cursor (important)

Recommandé — un seul Cursor, workspace suite

```
cd ~/Documents/Dev/Perso/Cloudity/Cloudity
cursor Cloudity.code-workspace
```

Tu vois **Cloudity** + **JobbingTrack** dans le même explorateur.
Pas besoin d'un second terminal `cursor .` pour JT.

Alternative — JT seul

```
cd ~/Documents/Dev/Perso/Cloudity/Cloudity/products/jobbing-track
cursor .
```

---

4. Orchestrateur ? Runtime ?

Aujourd'hui Cloudity **n'oblige pas** à démarrer JT.
Stacks Docker VPS restent **séparées** (cloudity / jobbingtrack-prod / preprod / gasoil / ytmusic).

« Orchestrateur » = cadre **dev + docs + submodules**, pas encore un seul `docker compose` unique.
C'est voulu pour **zéro perte** de données utilisateurs.

---

5. Données VPS (ne jamais Remove volumes)

Volume	~Taille
ytmusic_ytmusic_data	20,6 Go
gasoil_api_data	8,9 Go
jobbingtrack-prod_postgres_data	537 Mo
jobbingtrack-preprod_postgres_data	465 Mo
cloudity_postgres_data	49 Mo

Levier RAM : éteindre **JT preprod** hors usage (~15 conteneurs).

---

6. Prochaines étapes (propositions)

1. Travailler JT via `products/jobbing-track` + workspace.
2. Quand stable : submodule Gasoil → `products/fuel` (même procédure).
3. Puis PLM → `products/music` (volume 20 Go intact).
4. SSO Cloudity ID opt-in (platform/identity-sdk).
5. Maps plus tard.
6. Mises à jour : **par repo / par stack Portainer** ; éventuellement script « bump all submodules » sans redeploy forcé.

Versions API / images

Objectif : ≤ 5 tags d'écart vs prod ; éviter d'accumuler des images mortes (>1 Go/user de caches inutiles côté PLM déjà géré à part).
`docker image prune` **sans** toucher aux volumes.

---

7. Fichiers utiles

Fichier	Rôle
`docs/cursor/COMMENT-OUVRIR-CURSOR.md`	Ouvrir Cursor
`docs/cursor/BRIEF-INTEGRATION-SUITE.md`	Brief chat Cursor
`docs/ecosystem/ECOSYSTEME-SUITE-MODULAIRE.md`	Rapport long
`products/README.md`	Liste produits
`Cloudity.code-workspace`	Multi-root

---

8. Décisions encore à trancher (sans bloquer JT)

Naming Cloudity X vs -ity ; meta-repo séparé ou tout dans Cloudity.git (déjà engagé) ; premier SSO ; Maps dans Cloudity ou repo solo ; F-Droid ; éteindre preprod JT.

Comment ouvrir Cloudity + JobbingTrack dans Cursor

Ce qui est déjà en place (normal)

```
Cloudity/
├── Cloudity.code-workspace     ← ouverture « suite »
├── products/
│   ├── jobbing-track/          ← submodule Git = repo JobbingTrack
(complet)
```

```
│   ├── fuel/                    ← placeholder (Gasoil plus tard)
│   └── music/                   ← placeholder (PLM plus tard)
├── platform/identity-sdk/       ← SDK SSO (squelette)
├── backend/ frontend/ mobile/   ← Cloudity « cœur » (inchangé)
└── docs/cursor/BRIEF-….md       ← brief à donner aux chats Cursor
```

Oui, `products/` ne contient que JobbingTrack pour l'instant : c'est volontaire. Gasoil et PLM s'ajoutent **un submodule à la fois** (même procédure), sans toucher aux volumes Docker.

Le dossier historique `/Perso/JobbingTrack` peut encore exister : c'est le **même repo Git**.
Le chemin canonique dans la suite est désormais :

```
/Perso/Cloudity/Cloudity/products/jobbing-track
```

Deux façons d'ouvrir (pas besoin des deux)

A — Vue suite (recommandé si tu bosses Cloudity + JT)

```
cd /home/pactivisme/Documents/Dev/Perso/Cloudity/Cloudity
cursor Cloudity.code-workspace
```

Dans Cursor tu auras **2 racines** dans l'explorateur : 1. **Cloudity (suite)** — monorepo 2. **JobbingTrack** — `products/jobbing-track`

Tu **n'as pas** besoin d'ouvrir un second Cursor ni de faire `cd … && cursor .` dans un autre terminal :
tu changes juste de dossier dans le même workspace (Source Control = bon repo).

B — JobbingTrack seul (quotidien JT)

```
cd /home/pactivisme/Documents/Dev/Perso/Cloudity/Cloudity/products/jobbing-track
cursor .
```

Ou l'ancien chemin si tu préfères encore :

```
cd /home/pactivisme/Documents/Dev/Perso/JobbingTrack
cursor .
```

Orchestrateur : quoi démarre quoi ?

Besoin	Que lancer
Développer **seulement** JobbingTrack	Stack JT seule (`make` / compose JT) — **Cloudity n'est pas obligatoire**

Besoin	Que lancer
Développer **seulement** Cloudity Mail/Drive/…	Stack Cloudity seule
Tester **SSO / hub** plus tard	Cloudity + JT (quand OIDC sera branché)
Prod VPS aujourd'hui	Stacks Docker **séparées** (déjà le cas)

Cloudity n'est **pas encore** un orchestrateur runtime unique.
C'est un **cadre de développement + docs + submodule**. Les stacks Portainer restent indépendantes → **zéro perte** de données.

Après un commit dans JobbingTrack

```
cd products/jobbing-track
git push origin dev

cd ../..   # racine Cloudity
git add products/jobbing-track
git commit -m "chore: bump jobbing-track submodule"
git push
```

Brief pour un chat Cursor

Fichier à `@` : `docs/cursor/BRIEF-INTEGRATION-SUITE.md`

Brief Cursor — intégrer ce produit dans l'écosystème Cloudity

> **À coller / `@` dans un chat Cursor** ouvert sur JobbingTrack, GasoilTracking, YTMusic/PLM, ou Cloudity.
> Objectif : développer **dans** l'écosystème Cloudity (seul ou multi-root), **sans perdre** données utilisateurs ni casser Portainer.

---

1. Contexte en 10 lignes

- **Cloudity** = suite (auth, mail, drive, pass, agenda, notes, tasks, contacts, photos) + hub web.
  Chemin monorepo :
  `/home/pactivisme/Documents/Dev/Perso/Cloudity/Cloudity`
- **Satellites** (repos Git **dédiés**, aussi visibles sous `Cloudity/products/` en submodule) :
  - JobbingTrack → `products/jobbing-track/` (intégré en premier)

    - GasoilTracking → `products/fuel/` (à venir)
    - YTMusic/PLM → `products/music/` (à venir, **ne pas casser** le volume ~20 Go)
- On développe **soit** le produit seul, **soit** toute la suite via `Cloudity.code-workspace`.
- Déploiement VPS = **Portainer Git**, stacks **séparées**, volumes Docker **noms stables**.
- SSO « Cloudity ID » = **opt-in** plus tard ; auth locale de chaque app **reste**.

---

2. Comment ouvrir le projet dans Cursor

> Guide dédié (plus clair) : COMMENT-OUVRIR-CURSOR.md

Unitaire (recommandé au quotidien)

```
# JobbingTrack via submodule Cloudity
cd /home/pactivisme/Documents/Dev/Perso/Cloudity/Cloudity/products/jobbing-track
cursor .

# OU ancien chemin (tant qu'il existe encore)
cd /home/pactivisme/Documents/Dev/Perso/JobbingTrack
cursor .
```

Global (suite)

```
cd /home/pactivisme/Documents/Dev/Perso/Cloudity/Cloudity
cursor Cloudity.code-workspace
```

Les commits doivent aller dans le **bon repo** (submodule = repo JobbingTrack, racine = repo Cloudity).

```
# Depuis Cloudity monorepo : maj pointeur submodule après commit JT
cd products/jobbing-track
git checkout dev
# ... commits JT ...
git push origin dev
cd ../..
git add products/jobbing-track
git commit -m "chore: bump jobbing-track submodule"
git push
```

---

3. Règles absolues (données & prod)

Faire	Ne jamais faire
Backup volume avant manip VPS	`docker compose down -v`

Faire	Ne jamais faire
Redeploy Portainer **sans** Remove volumes	Renommer un volume Docker à la volée
Garder stacks séparées (cloudity / JT prod / JT preprod / gasoil / ytmusic)	Fusionner toutes les DB dans un seul Postgres « pour simplifier » sans plan
SSO opt-in	Forcer migration comptes existants
Améliorer Cloudity **en s'inspirant** du code JT (patterns, monitoring)	Copier-coller half-baked JT dans `backend/` Cloudity

Volumes critiques (VPS Contabo) — audit 2026-09-15

Volume	Taille	Produit
`ytmusic_ytmusic_data`	**~20,6 Go**	PLM / YTMusic
`gasoil_api_data`	**~8,9 Go**	GasoilTracking
`jobbingtrack-prod_postgres_data`	~537 Mo	JT prod
`jobbingtrack-preprod_postgres_data`	~465 Mo	JT preprod
`jobbingtrack-prod_mobile_releases`	~1,2 Go	OTA JT
`cloudity_postgres_data`	~49 Mo	Cloudity
`cloudity_mobile_data`	~678 Mo	Cloudity mobile

Détail décisions : `../ecosystem/EMAIL-PORTEUR-DECISIONS-SUITE-2026-09-15.md`.
Ops Cursor/Portainer : `../ecosystem/ARCHITECTURE-CURSOR-PORTAINER-SUITE.md`.

Volumes critiques (VPS Contabo) — (ancien résumé)

- JobbingTrack prod : `jobbingtrack-prod_postgres_data`
- JobbingTrack preprod : `jobbingtrack-preprod_postgres_data`
- Gasoil : `gasoil_api_data` (~9 Go)
- PLM/YTMusic : `ytmusic_ytmusic_data` (~21 Go)
- Cloudity : `cloudity_postgres_data`, `cloudity_mobile_data`

---

4. Ce que « intégrer dans Cloudity » veut dire (et ne veut pas dire)

Oui

1. Le code JT est **clonable / ouvrable** sous `Cloudity/products/jobbing-track` (submodule).
2. Workspace Cursor multi-root.
3. Plus tard : tuile hub Cloudity + OIDC + éventuellement Mail/Agenda (scopes).

4. Réutiliser de JT vers Cloudity : idées monitoring, Makefile ops, patterns microservices — via **extraits / libs**, pas un big merge.
5. Mises à jour **par produit** (image GHCR / tag) avec un **orchestrateur** éventuel (Watchtower filtré, ou script `suite-update`) qui respecte chaque app.

Non

1. Un seul conteneur monolithique.
2. Perdre preprod JT ou forcer une seule DB.
3. Casser le fonctionnement actuel de YTMusic/PLM « pour aligner ».
4. Plus de 5 versions d'API **lourdes** en parallèle sur le serveur sans budget disque/RAM (voir §6).

---

5. Apports croisés (quoi prendre de chaque app)

Source	Apport potentiel à Cloudity / suite
JobbingTrack	Beaucoup de microservices Node ; monitoring ; preprod+prod ; mobile Flutter ; triage email (à brancher plus tard sur Cloudity Mail)
GasoilTracking	Trajets GPS, UX mobile Expo → alimentera **Cloudity Maps**
YTMusic/PLM	Offline, cache média, OTA — **garder le comportement** ; brancher SSO progressivement
Cloudity	Auth JWT/2FA, gateway Go, design system `@cloudity/ui`, hub apps

---

6. Serveur Contabo — perf & versions (constat + reco)

Mesures indicative (2026-09-15) : **~29 Go RAM**, **~7 Go used**, disque **~43 %** (478 Go / 1,2 To).
JobbingTrack prod + preprod = double flotte de services (~60–80 conteneurs suite+JT) → **gros levier** : éteindre preprod hors usage, mutualiser Redis/limites mémoire.

Recommandations Cursor/ops :

1. Ne pas lancer preprod JT 24/7 si inutile.
2. Limiter tags d'images API gardés sur le VPS (≤ 5 tags « actifs » hors prod, sauf si cache user > 1 Go/user → purge).
3. Watchtower / updates : **labels par stack**, jamais un pull all destructif.
4. YTMusic : garder limite mémoire (~3 Go déjà) ; ne pas dupliquer le volume data.

---

7. Checklist agent Cursor (quand on te demande d'avancer)

1. Confirmer le repo ouvert (Cloudity racine vs `products/jobbing-track`).

2. Ne pas toucher aux volumes / `.env` prod sans backup.
3. Si changement JT : commit **dans le submodule**, puis bump pointeur dans Cloudity.
4. Si changement suite Cloudity : ne pas casser les builds GHCR existants.
5. Toute idée SSO / Mail : feature flag / opt-in.
6. Documenter dans `docs/` du produit concerné, pas un nouveau roman à la racine GitHub.

---

8. Fichiers de référence

Fichier	Rôle
`docs/ecosystem/ECOSYSTEME-SUITE-MODULAIRE.md`	Rapport long (aussi en PDF)
`products/README.md`	Carte des submodules
`Cloudity.code-workspace`	Ouverture multi-root
`docs/INDEX.md`	Index doc Cloudity
`products/jobbing-track/docs/SUITE-ECOSYSTEM-LINK.md`	Fiche satellite JT

---

9. Prochaine étape humaine (pas encore codée)

- Décisions naming / SSO / Maps (voir rapport PDF).
- Submodule Gasoil + PLM quand JT est stable dans le workspace.
- Script VPS `backup-suite-volumes.sh`.
- identity-sdk réel.

Fin du brief — garder ce fichier à jour si la structure `products/` évolue.